BYZ652 Software Architectures

Term Project

- Fundamentals -

Understanding Software Architecture, Quality, and Security Through Code Analysis

Jay Marchetti

Software Engineering Institute

Carnegie Mellon University

Code Analysis

These are not three independent concepts…

- In fact they are highly inter-related.

Code analysis - from a technical viewpoint…

- Why is it important?

Code analysis can provide key insights into your software's:

- Architecture
- Quality
- Security

✓ Architecture is the fundamental organization of a system embodied in its components, their relationships to each other, and to the environment and principles guiding its design and evolution.

✓ Quality is the software's system-specific fitness for purpose, along with the underlying structural characteristics that make it so.

✓ Security is the specific quality characteristic for thwarting attacks and is key for today's operational success.

Code Analysis: The Technical Context

What is "code analysis"?

- **Code analysis** is the automated analysis of software to assess the degree to which it embodies one or more of the desired **quality attributes**.

- A quality attribute (QA) is "a measurable or testable property of a system that is used to indicate how well the system satisfies the needs of its stakeholders" [2].
- Certain QAs will be of high importance for a given system. For example:
  - performance
  - interoperability
  - safety
  - maintainability
  - security
  - testability
- While others may not be important for the system at hand, e.g.:
  - portability
  - scalability

Code Analysis: The Technical Context

Quality Attributes from code?

The architecturally significant Quality Attributes (QAs) drive the system's architecture.

- But QAs can also be viewed as *aggregates of code characteristics* that can be tested for at runtime or identified within the source code.

However, most QAs are very difficult and expensive to add later.

- So, we apply **code analysis** early on and verify that the essential QAs have been designed into the system's "ground truth" artifact: the code.

TAKEAWAY

- Use code analysis early in the software development life cycle to verify essential QA's.
- Code analysis may sometimes be done "after the fact" to assess (or *audit*) whether code exhibits the desired (or *required)* QAs.

Quality Attributes Drill Down - Examples

QA	Meaning / Relevance [2]
Usability	Ease with which the user can accomplish a desired task
Availability	Ready to perform whenever needed; encompasses reliability
Performance	Ability to meet system timing requirements
Testability	Ease of demonstrating the software's faults through testing
Modifiability	Minimized cost and risk of making changes to the software
Interoperability	Ease of exchanging information via interfaces with other systems
Safety	Avoidance of states that may damage or cause injury to external actors
Integrity	Assurance that information is accurate, complete, and valid, and has not been altered by unauthorized actions

Quality Attributes - Impacts to Software

Quality Attributes impact your system's architecture, code quality, and code security.

- Code analysis can be used to verify many high-priority QAs.

Impacting	Example Relevant QA
Architecture	Scalability, modularity, interoperability, safety
Code quality	Maintainability, testability, portability, safety
Security	Confidentiality, integrity

Question: The QAs below would most likely impact which: architecture, code quality, or security?

Modifiability → Code Quality?

Reliability → Architecture?

Availability → Security?

An argument could certainly be made that all 3, architecture, code quality, and security, are impacted by these QA's.

The Two Types of Code Analysis

There are two primary types of code analysis: **static** and **dynamic**.

Static analysis is the analysis of software while it is not executing. It is defect prevention.

Dynamic analysis is the analysis of software while it is executing. It is defect detection.

STATIC ANALYIS (SA)	DYNAMIC ANALYSIS (DA)
• Includes analyzing binaries, bytecode, or - most commonly (and our focus here) – the source code.	• Includes runtime monitoring, fuzzing, and unit, integration, and system testing.

Use both ☆ Example: Array bounds violation - Lint (SA tool) may find it via data flow analysis of source code, Purify (DA tool) may find it via tagged memory run-time instrumentation. Using both SA and DA increases the odds it is not missed!

Software Quality – Embodying the QAs

So again: What is software quality?

Software Quality is the degree to which software embodies the required QAs, thus indicating whether the it meets stakeholder needs [2].

External QAs (Execution)		Internal QAs (Evolution)	
Correctness	Usability	Maintainability	Flexibility
Efficiency	**Reliability**	Portability	Reusability
Integrity	Adaptability	Readability	**Testability**
Accuracy	Robustness	Understandability	Scalability
Performance	Availability	Interoperability	Deployability

TAKEAWAY

- External QAs are evident to end users, internal QAs are evident mainly to developers.
- Internal QAs often affect external QAs: if it isn't **testable**, how do you know it's **reliable**?

Software Quality – Throughout the SDLC

Software quality cannot be bolted on – it must be built in

Software quality is a measure of how well software performs the function for which it was intended, including:

- Performing the intended job both initially and throughout the Software Development Life Cycle (SDLC)
- Encompassing all of the relevant QAs, both external and internal

While code analysis is the main subject of this course, we strongly assert this **Best Practice:**

High-quality software starts with high-quality requirements and extends through architecture, code development, documentation, testing, deployment, and sustainment.

TAKEAWAY

- High-quality software meets or exceeds all aspects of its intended purpose.
- You build quality into software throughout the Software Development Life Cycle.

Software Quality: A Common (but Wrong) Hierarchy of Priorities

Warning: The considerations that drive programs too often fall into 3 classes of priority…

Consideration	Priority	Overarching Associated Questions
Operationality	High	Does it work? Are all behavioral requirements met?
Cost and schedule	High	Will it be within budget and meet the targeted schedule?
Safety- and mission-criticality	Mid	Are all safety requirements and certifications met? Are all foreseeable unsafe or other critical failures mitigated?
Security	Mid	Are all IA requirements and certifications met? How difficult is it to exploit?
Non-behavioral requirements	Low	Does it incorporate the QAs, both external and internal, that have been deemed most important for the system?

TAKEAWAY

- Singular focus on the high- and mid-level priorities may gain short-term benefits, but at very high expense to longer-term costs.
- Strict adherence to these priorities often portends program failures downstream.

Coding Standards

A **coding standard** provides a set of coding practices for use by the development team to enhance source code quality.

Coding Standard Type	Intent	Usage
Stylistic	Project and / or organizational conventions for consistent, readable, and maintainable code.	Most modern organizations have adopted stylistic guidelines for all new code, which their developers are strongly encouraged (or required) to use.
Critical System	Language restriction rules for secure, safety-critical, and mission-critical code.	Rule-based standards are important for systems where security or criticality are key, but they demand additional investment in developer training and tools.

TAKEAWAY

- Stylistic coding conventions are a best practice that should always be used.
- Additional standards for criticality and security are essential for some systems.

Why are Stylistic Coding Standards Important?

As a new hire, you're assigned to maintain one of your company's two software products. Each is 2.5 MSLOC of C++ code. Below are code snippets typical of each:

Product A

```
1  void main(int argc, char* argv[]) {
2      while (1){ int in;bool cit[10];
3        cin>>in; cit[0]=0;
4           for (int i =1;i<in; ++i) cit[i]= 1;
5        if (!in) exit(-1) ;int m=1; int cp=0;
6          while(oneleft(cit,in)!=12)
7          { if(oneleft(cit,in)==-2){
8          ++m; cp=0;cit[0]=0;
9                  for (int i=1;i<in;i++) cit[i]=1; }
```

Product B

```
1  // Main - Start and initialize the system
2  void main(int argc, char* argv[]) {
3      bool bDone = false;
4      bool bMotorInitialized = false;
5      while (bDone == false)
6      {
7          // Start watchdog timer
8          int nRetVal = nInitWDog(WDOG_PERIOD_MSEC);
9          if (nRetVal != NO_ERROR)
10         {
11             cout << "Watchdog init failed with ID = " << nRetVal << endl;
               rocessing incoming commands until STOP issued
               and = nReadCmdChannel();
               and == STOP)
               = true;
```

Secure Coding Standards

- **SEI CERT** Secure Coding Standards for C, C++, Java, and Perl
- **CWE:** The MITRE Corporation's Common Weakness Enumeration for C/C++
- **OWASP:** Open Web Application Security Project

Clean Code!

Your assignment entails adding new features and fixing reported bugs.
Question: On which product would you likely be most productive?

Answer: B

Static Analysis: Verifying Code *in vitro*

Static Analysis (SA) is the analysis of software while it is not executing.

SA may be done on Java or .NET bytecode or on binaries, ordinarily to address security concerns about the software supply chain.

- However, SA most often implies analysis of source code, which is our focus.

SA typically means automated source code analysis, i.e. using tools, though manual analysis – or peer review – is a best practice that may be considered a form of SA.

- This presentation targets source code analysis using SA tools.

TAKEAWAY

- SA can be done on bytecode, binaries, or (most commonly) on source code.
- SA is generally automated via the use of one or more tools.

Static Analysis: Considerations for Choosing SA Tools

Consideration	Scope
License pricing	Free to ~$50K per project
Languages supported	C, C++, C#, Java, Python, Ada, other
Coding standards supported	MISRA, JSF AV++, HIC++, CERT Secure Coding, other
Intended usage environment	DevOps/CI, GUI, command line, in - IDE, browser dashboard
Focus of SA concern	Code quality, safety, security, 64 - bit, other
Buildable system required	Complete build environment needed to only source code needed
Tool reputation	Small project on GitHub to Coverity
Country of origin	U.S. or U.S. allies, Russia, others
Ease of configuration and use	Simple to guru - friendly

TAKEAWAY

- No SA tool is right for all situations. A variety of considerations must be addressed.

Static Analysis: A Sampling of Well-Known SA Tools

Tool	Vendor	Notes
Understand	Scientific Toolworks	Swiss-army knife SA tool, works on non-buildable source, supports many languages and MISRA
Coverity	Synopsys	Top-line SA tool, very expensive, many languages and standards
PC-lint	Gimpel Software	Venerable command-line Lint tool, C/C++ only, MISRA
PolySpace	MathWorks	Bug Finder and Code Prover, C/C++, many criticality standards
FindBugs	Open source	Well-known FOSS tool, Java only
CppCheck	Open source	Well-known FOSS tool, command line or GUI, C++ only
Lattix	Lattix	Special purpose tool for architectural views of C++ / Java code
Eclipse	Open source	IDE with many SA plugins available for C++ / Java code

TAKEAWAY

- Good SA tools abound, though SEI does not endorse specific tools.

Metrics

Metrics are measures or counts of certain aspects of code that **provide insight into a variety of code QAs**.

Metric	Relevant QAs
Comment-to-Code Ratio	Descriptiveness, Modifiability
Maximum Nesting Depth	Understandability, Complexity
Average Dependency	Coupling, Reusability

Static Analysis (SA) tools typically support a wide variety of metrics. Here are just a few examples.

We'll cover **two** base metrics all developers and acquirers should know for their system:

Code Volume	Code Complexity

TAKEAWAY

- Experienced developers review metrics for their code as a tool to improve it.
- Acquirers should request regular metrics reports from contractors to put an SA deliverable up front.

Metrics: Code Volume

Source Lines of Code (SLOC) is a rule-based count of the number of lines in a program's source code. It is the most familiar code volume metric.

Two SLOC types with differing counting rules are most often cited: physical and logical SLOC:

Physical SLOC	Logical SLOC
Line count ignoring blank and comment lines	Logical statements (e.g., semicolon terminated)
Sensitive to code formatting and layout style	Insensitive to code formatting and layout style
Not language-specific	Language-specific counter program is required
Typically (2–3x) greater than logical SLOC	

TAKEAWAY

- When assessing SLOC, be sure you understand the counting rules being used.

Metrics: SLOC Misuse and Use

SLOC is sometimes controversial because it can be misused.

Misuse	Use
Misusing SLOC as a productivity metric incentivizes code bloat and fails to capture differences in: • Language – ASM versus C/C++ • Code complexity – Naturally complex code • Developer expertise – The best developers may actually write fewer lines of (superior) code and / or be assigned more complex modules	Correlates with total system complexity and many other related and important concerns [6], e.g.: • Maintenance costs • Migration costs • Undetected error counts • New developer training time • New feature costs • Required development resources

Key Idea: SLOC is akin to mass. "The greater the SLOC the more inertia to change the system will have, the more people it will take to keep it fed, and the more stakeholders who will be crawling all over it."[Grady Booch]

TAKEAWAY

- Never use SLOC in the context of developer or contractor productivity.
- SLOC correlates with many key concerns for both new and existing code.

Metrics: Complexity

There are two ways a software-intensive system exhibits complexity.

- Both render the system difficult to manage, maintain, modify, reuse, or test.

Code Complexity

The most common metric for code complexity is McCabe's cyclomatic complexity (CC), which is a count of the number of linearly independent paths through the code.

Architectural Complexity (AC)

A relative measure of the number of classes/objects, processes/threads, components, and nodes, along with the quantity and type of interconnections between them [6].

Key Idea: Software Entropy → Without energy input (i.e., maintenance, refactoring), a system's complexity will increase over time toward chaos, with respect to both architecture and code.

- This accumulation of disorder over time is one form of Technical Debt.

Metrics: Code Complexity

The more typical use of the term *complexity* with respect to software relates to the structural complexity of the source code itself, rather than the program's architecture.

There are two well-known code complexity metrics:

- **Cyclomatic Complexity (CC):** Proposed by McCabe (1976) to support basis path (aka structured) testing. This is, by far, the most used code complexity metric.
- **Halstead Complexity (HC):** From his "software science" work, halstead metrics are based on number of code operators and operands and go well beyond just complexity.

  Coding standards for critical systems usually stipulate limits on CC for all functions. Why? Because Low CC correlates with **testability**.

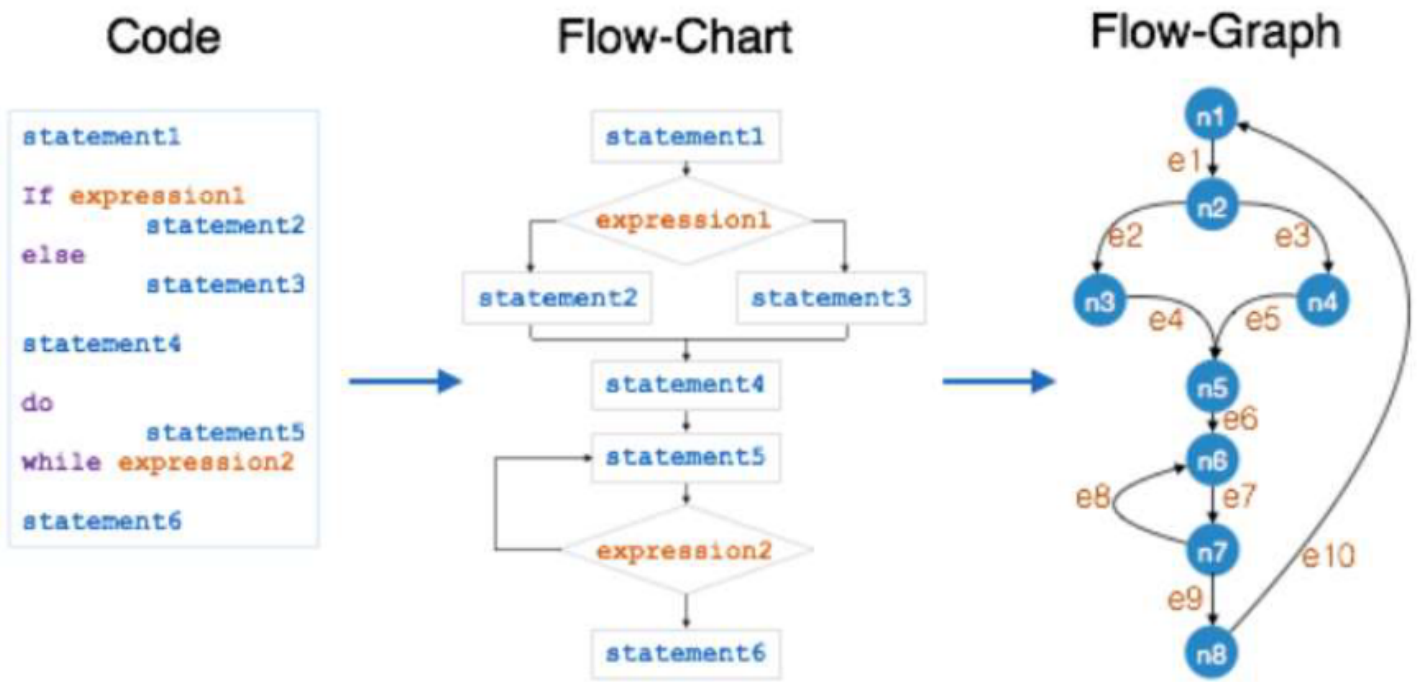
TAKEAWAY

- CC is the most common code complexity metric.
- Many standards for critical systems place strict upper limits on CC.

Metrics: Cyclomatic Complexity Measures Code Testability

CC is the number of linearly independent paths through the code and is based on the code's control flow graph (CFG).

CC may be calculated for an entire application, but more usefully it is calculated for a function module since CC equals the number of test cases required to unit test each independent code path [9]. This is also known as Basis Path testing.



$v(G) = e - n + 2$
where [9]
e = # flow-graph edges
n = # flow-graph nodes

Here:
$e = 10$
$n = 8$
CC = 10 – 8 + 2 = 4

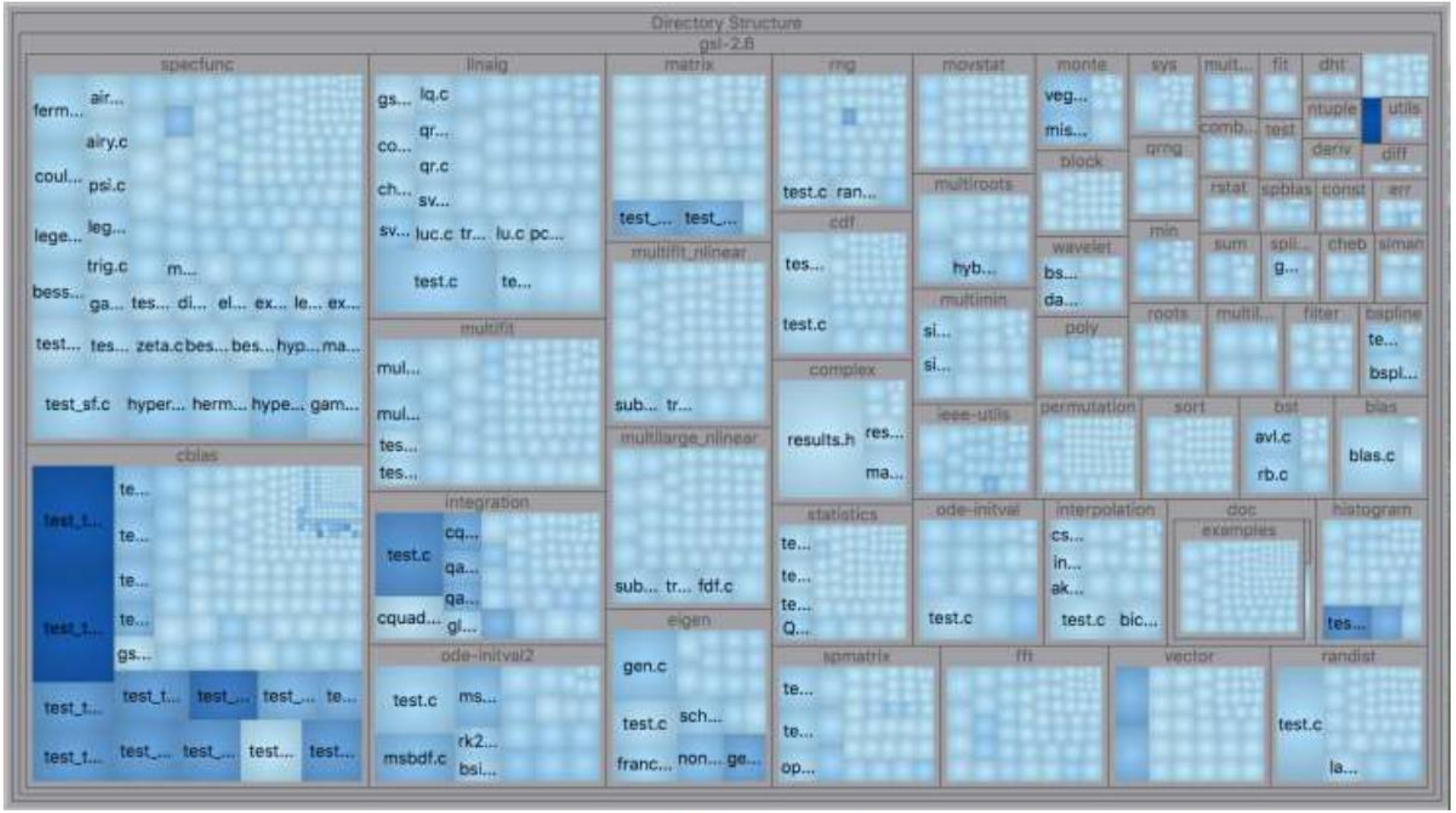
Shortcut: CC = # CFG enclosures + 1

Cyclomatic Complexity – Treemap Visualization

The treemap visualization shows all files in the code base and their relative CC.

Here cell size indicates each file's SLOC and **blueness** indicates its maximum function CC.

Code base: GNU Scientific Library



Treemap Created Using SciTools Understand

Metrics: Complexity + Commits = Hot Spots

While "scholarly data on code complexity metrics is mixed because of the difficulty of performing controlled studies across different application areas, different programming team skill levels, different languages, and other confounding factors." [Koopman]

Key Idea: Cyclomatic Complexity (CC) remains a useful **signal** for code that is difficult to understand, difficult to test, and more prone to defects.

However, defect prediction can be further enhanced by incorporating the commit history data from a version control system (VCS) such as Git.

- A code defect **hot spot** is the intersection of high-complexity code and high code-change frequency.
- Hot spots identify difficult code that has been worked on extensively. Research has shown such code to have a higher likelihood of latent defects [11].

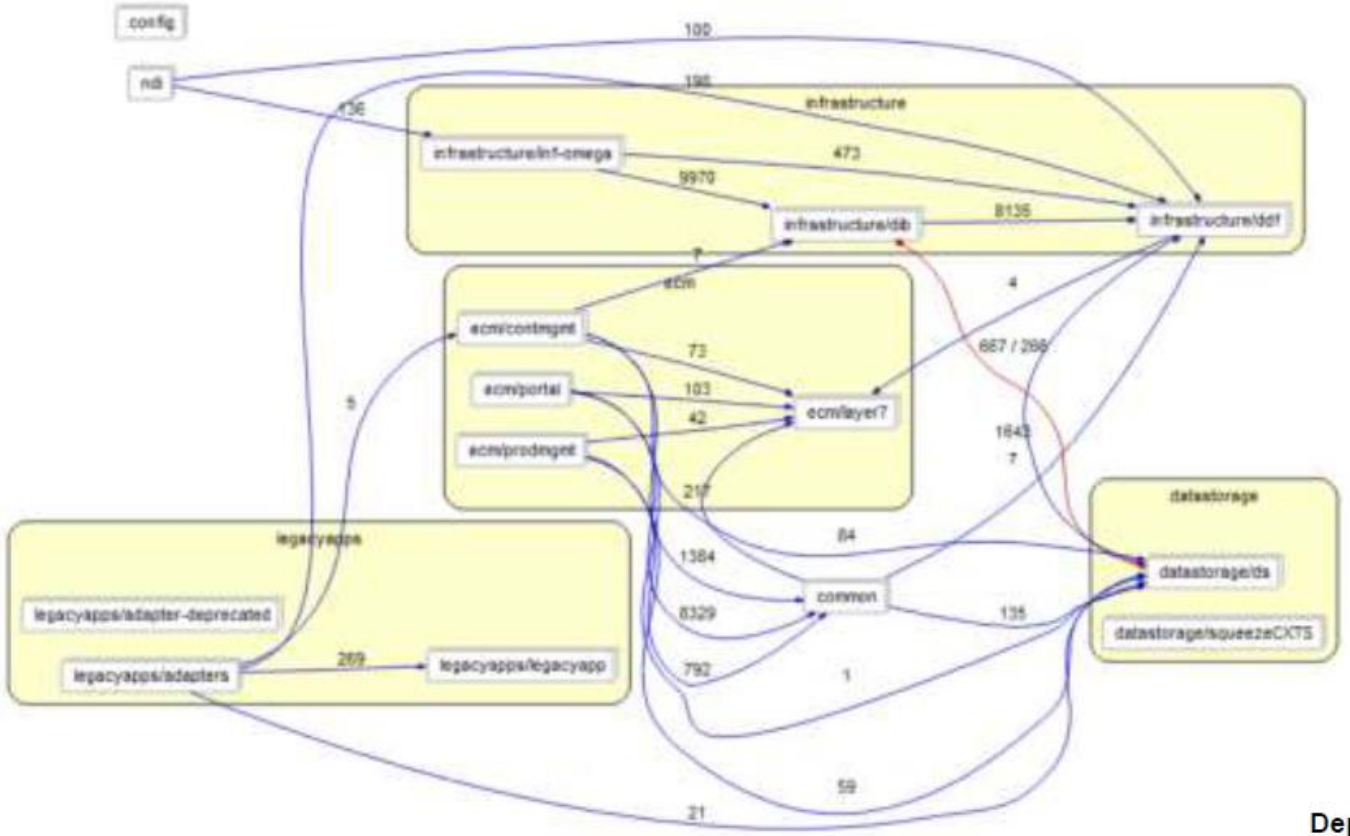
TAKE AWAY

- Using code complexity metrics plus VCS history shows promise for bug prediction.

Metrics: Architectural Complexity – From Code Analysis?

Given a legacy system, where all we have is the codebase, is there any aspect of architectural complexity (AC) that is evident purely from SA tools?

- Yes, we can look at the number and types of interdependencies among modules or classes using dependency graphs.

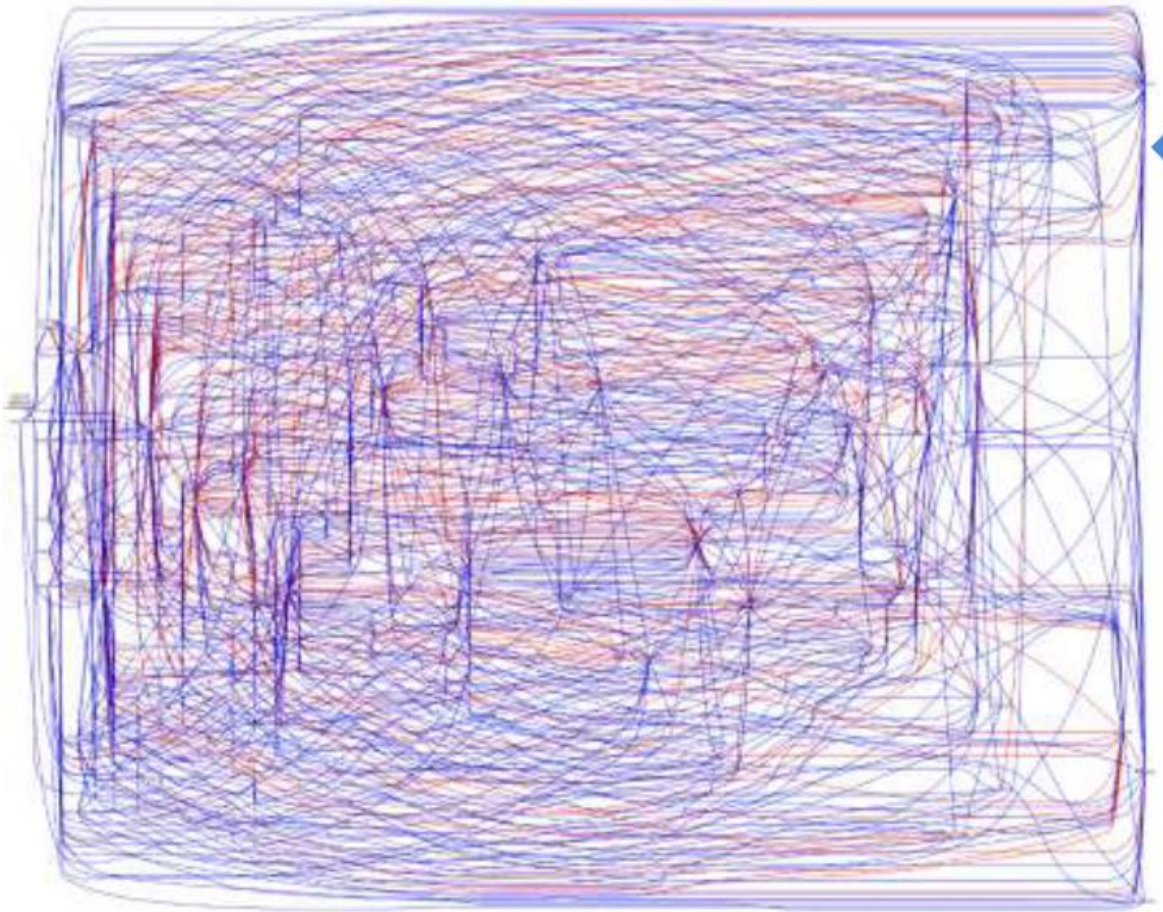


For small to medium systems, dependency graphs provide:

- Views of the main subsystems and their components
- Indications of the number of couplings between components and whether any are bidirectional (red)

Dependency Graph – Created Using SciTools Understand

Architecture – From Code Analysis



Dependency Graphs

For large or complex systems, dependency graphs quickly get out of hand.

Yet, given a code base we need *some* insight into its architecture to:

- Effectively change or maintain the system.
- Assess potential reuse of system components.

But reengineering architecture from code is a hard problem that's not fully automatable.

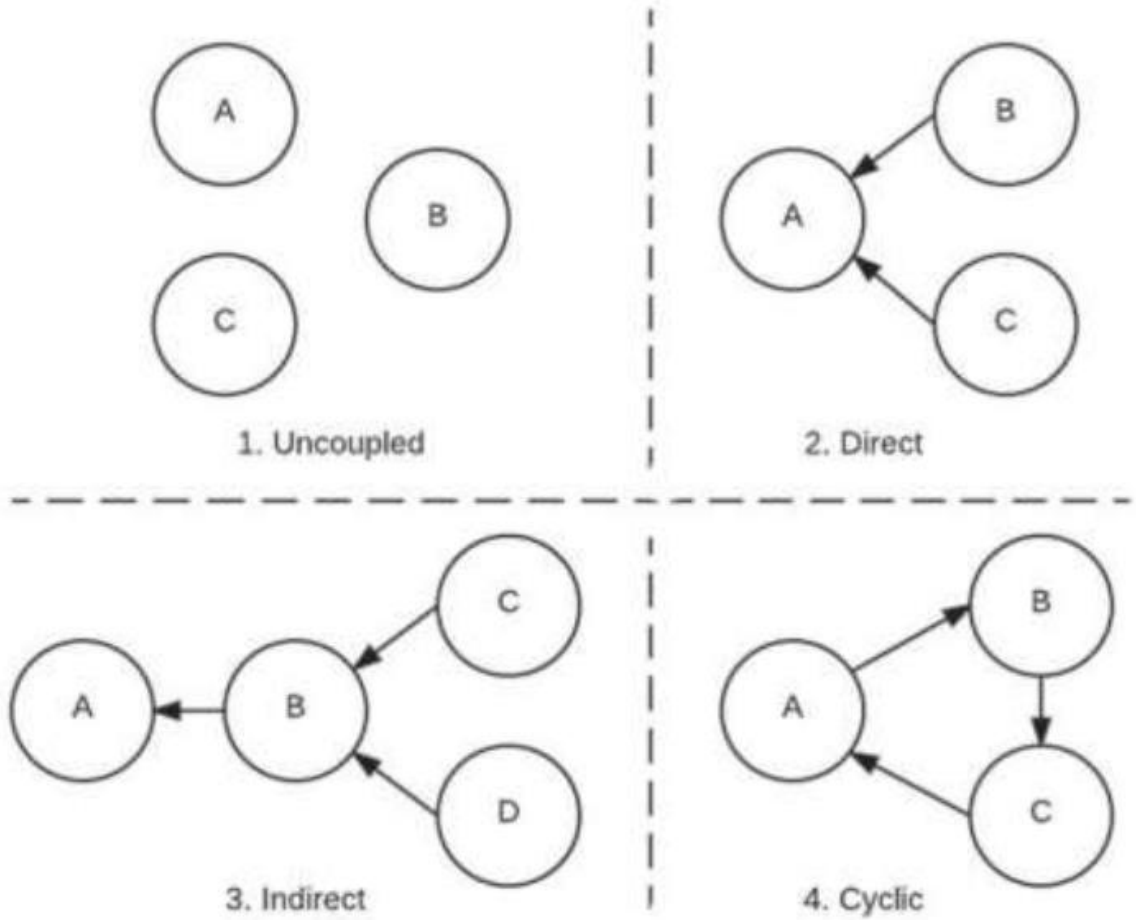
TAKEAWAY

- Extracting architecture from code analysis for large or complex systems is analyst-intensive.
- SA tools help, but no current tool can automate the process.

Architecture – Dependency Types

Modules within a system may be dependent in several ways:

1. Uncoupled
2. Directly coupled
3. Indirectly coupled
4. Cyclic

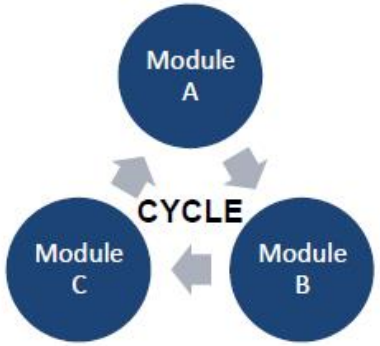


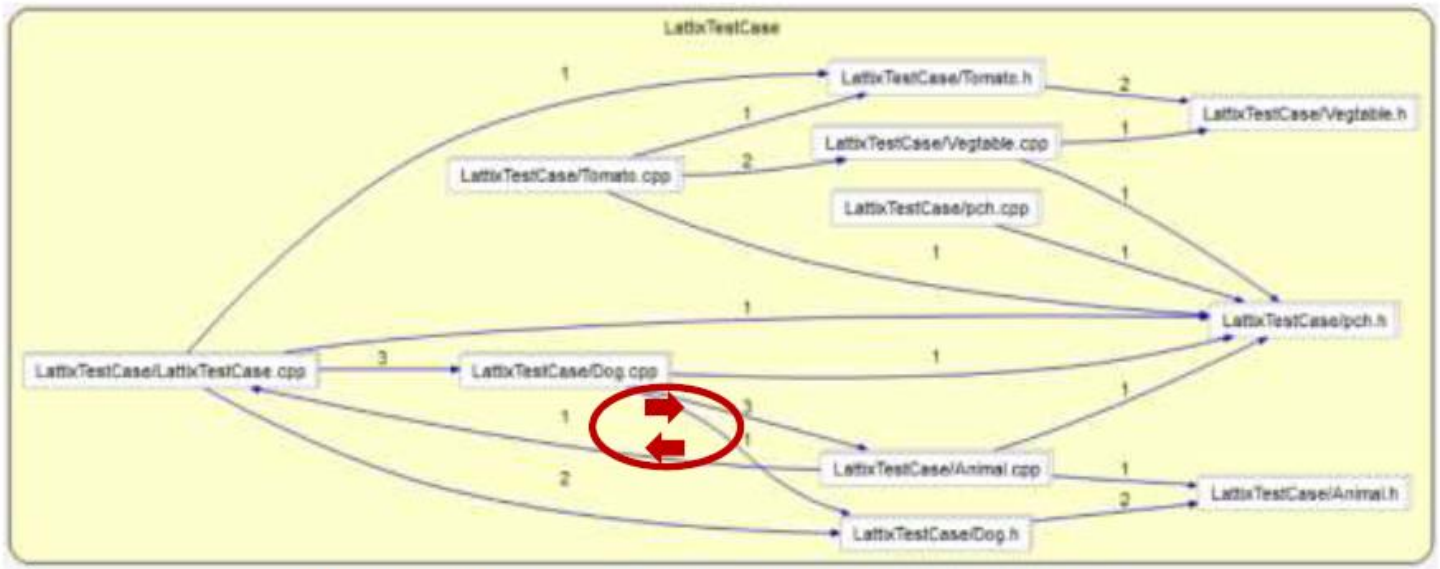
Adapted from [15]

Architecture – DSM and Cyclic Dependencies

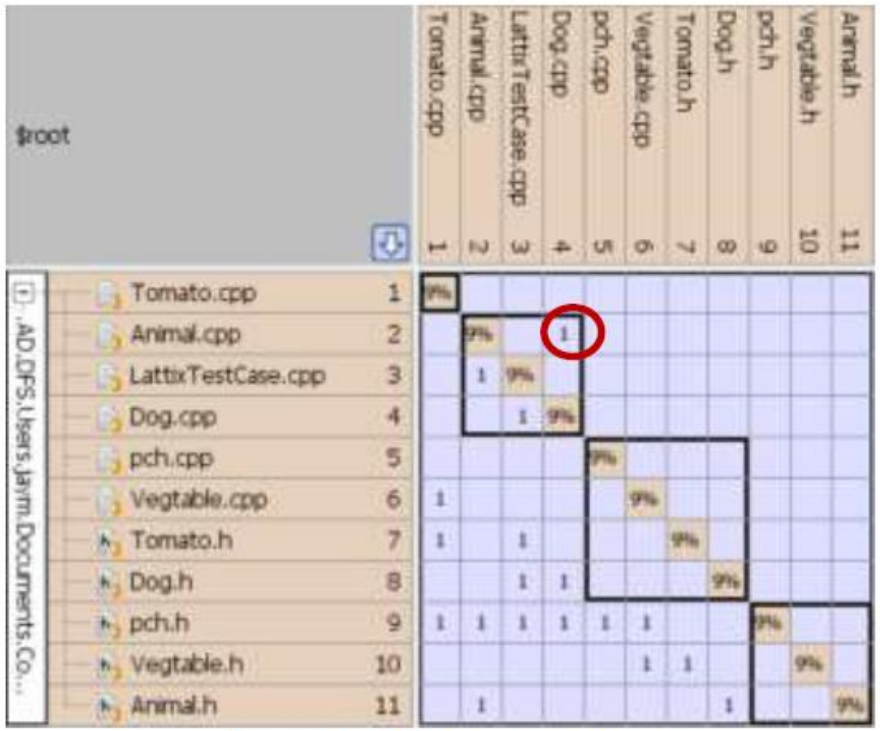
A good dependency visualization for large or complex systems is the Dependency Structure Matrix (DSM).

DSM's show cyclic dependencies (aka "cycles") as entries above the diagonal.





Dependency Graph – Created Using SciTools Understand

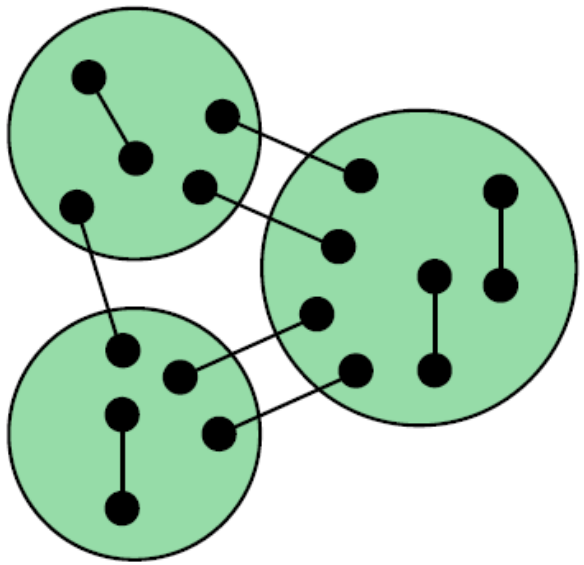


DSM – Created Using Lattix Architect

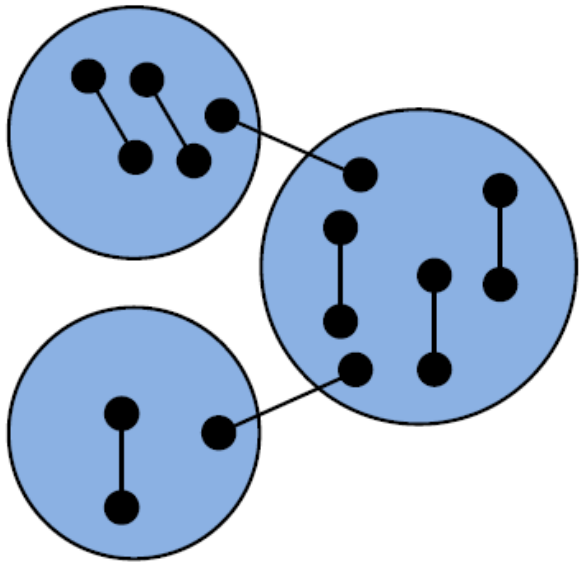
Two Dependency Views of the Same System containing a Cycle

Architecture – Coupling versus Cohesion

Static Analysis can reveal the coupling and cohesion levels within a system.



System A



System B

Example at left [Adapted from 7]: Circles are modules, lines are calls, and line ends are functions.

- Cohesion: calls inside a module of unified concept
- Coupling: calls between modules

Q: Which system is easier to modify and maintain?

A: System B

	System A	System B
Cohesion	Low	High
Coupling	High	Low

Code Quality Issues: Typical Findings

Detected by Tools		Detected by Analyst
Magic numbers	Ignored return values	Watchdog-timer use poor or lacking
Uninitialized variables	Assembly language	Homegrown RTOS or database
Dead code	Conditional compilation overuse	Timing / memory not instrumented
Commented-out code	High complexity code	Poor error handling / logging
Vulnerable library calls	Cyclic dependencies	CPU, OS, or database dependencies
Unvalidated inputs	Build warnings	Poor concurrency management
Global variables	Poor modularity / high coupling	Ad-hoc / incoherent architecture
Stylistic inconsistency	Duplicated code	
Lack of commenting	High SLOC functions and files	

Code Quality Issues: Examples

```
1   #include <string.h>
2
3   int G;
4
5   size_t Function1(int a, void* vptr, char* userinput) {
6       float* ptr1, ptr2;
7       float x, a = 6.023E+23;
8       char s[12];
9       ptr1 = (float *)vptr;
10      if (*ptr1 > a)
11      {
12          x = x + 3.1415927 * a + (float)G;
13      }
14      else {
15          // G = a;
16          strcpy(s, userinput);
17      }
18      return strlen(s);
19  }
```

	Issue
●	Magic numbers
●	Uninitialized variables
●	Dead code
●	Commented-out code
●	Vulnerable library calls
●	Unvalidated inputs
●	Global variables
●	Stylistic inconsistency
●	Lack of commenting